

Chapter 5

Glue Types and Univalence

同値な型は等しいか。たとえば Theorem 76 の $\mathbb{Z}$ と Int は、互いに逆な写像で結ばれているという意味で「同じ型」であるが、この 2 つを結ぶ $\text{Path type}_i \mathbb{Z} \text{Int}$ を作る手段を、本書はまだもっていない。 $\langle i \rangle$ で型どうしの path を作るには、 i に依存する型の式が要るが、これまでの型構成子はどれも「両端で同じ構成子」の線しか作れないからである。本章で導入する *Glue* 型は、まさにこの手段を与える：同値 $e : T \simeq A$ を「境界の材料」として型の線を作る構成子であり、そこから *univalence*—同値ならば path で結ばれる—が定理として従う。

まず同値の概念を定義する。

Definition 87 (Contractibility, Fibers, and Equivalences). $A, B : \text{type}$, $f : A \rightarrow B$, $b : B$ に対して：

$$\begin{aligned} \text{isContr}(A) &\stackrel{\text{def}}{=} (x : A) \times (y : A) \rightarrow \text{Path } A \, x \, y \\ \text{fib}(f, b) &\stackrel{\text{def}}{=} (x : A) \times \text{Path } B \, (f \, x) \, b \\ \text{isEquiv}(f) &\stackrel{\text{def}}{=} (b : B) \rightarrow \text{isContr}(\text{fib}(f, b)) \\ A \simeq B &\stackrel{\text{def}}{=} (f : A \rightarrow B) \times \text{isEquiv}(f) \end{aligned}$$

$e : A \simeq B$ の関数成分を $\pi_1(e)$ と書く。

$\text{isContr}(A)$ は「 A には元があり、他のすべての元がそれと path で結ばれる」— A が 1 点に潰れる—ことをいう。 $\text{fib}(f, b)$ は b の逆像（とその witness）であり、 $\text{isEquiv}(f)$ は「すべての逆像がちょうど 1 点」、すなわち f が全単射の homotopy 版であることをいう。恒等写像が同値であること ($\text{idEquiv}(A) : A \simeq A$) は、 $\text{fib}(\text{id}, b) = (x : A) \times \text{Path } A \, x \, b$ が singleton であり、その可縮性を Section 2.2 で示してあることから従う。また、互いに逆な写像の組（往復がそれぞれ path で恒等に等しいもの）から isEquiv が構成できることが知られている（証明は The Univalent Foundations Program 2013 の §4.4 を参照。以下この事実を用いる）。これにより、Theorem 76 で示した往復の path の組は、正式に $\mathbb{Z} \simeq \text{Int}$ の元を与える。

Definition 88 (Glue Types).

$$\frac{\Gamma \vdash \varphi : \mathbb{F} \quad \Gamma \vdash A : \text{type} \quad \Gamma, \varphi \vdash T : \text{type} \quad \Gamma, \varphi \vdash e : T \simeq A}{\Gamma \vdash \text{Glue}[\varphi \mapsto (T, e)] A : \text{type}} \text{ (GlueF)}$$

$$\frac{\Gamma, \varphi \vdash t : T \quad \Gamma \vdash a : A \quad \Gamma, \varphi \vdash \pi_1(e) t = a : A}{\Gamma \vdash \text{glue}[\varphi \mapsto t] a : \text{Glue}[\varphi \mapsto (T, e)] A} \text{ (GlueI)}$$

$$\frac{\Gamma \vdash b : \text{Glue}[\varphi \mapsto (T, e)] A}{\Gamma \vdash \text{unglue } b : A} \text{ (GlueE)}$$

さらに、次の ∂ -等値性が成り立つ：

$$\Gamma, \varphi \vdash \text{Glue}[\varphi \mapsto (T, e)] A =_{\partial} T : \text{type}$$

$$\Gamma, \varphi \vdash \text{glue}[\varphi \mapsto t] a =_{\partial} t : T \quad \Gamma, \varphi \vdash \text{unglue } b =_{\partial} \pi_1(e) b : A$$

また、 β 簡約と η 規則：

$$\text{unglue}(\text{glue}[\varphi \mapsto t] a) \rightarrow_{\beta} a \quad (\beta)$$

$$\text{glue}[\varphi \mapsto b](\text{unglue } b) = b : \text{Glue}[\varphi \mapsto (T, e)] A \quad (\eta)$$

直観的には、 $\text{Glue}[\varphi \mapsto (T, e)] A$ は「extent φ の上では T 、それ以外では A 」を、同値 e で貼り合わせた型である。第 1 の ∂ -等値性が「 φ 上では T そのもの」であることを述べ、 unglue は貼り合わせを剥がして A 側の成分を取り出す（ φ 上では b 自身が T の元なので、第 3 の ∂ -等値性のとおり e で送った $\pi_1(e) b$ が A 側の成分である）。（ GlueI ）の第 3 前提は貼り合わせの整合性— T 側の材料 t を e で送ると A 側の材料 a に一致すること—を要求する。なお $\beta \cdot \eta$ は導入と除去の detour に関する規則であり、Section 3.6 の分類がここでも維持されていることを確認してほしい。

Glue 型の κ -簡約 (hcomp と transp) は本体系で最も込み入った部分であり、その構成には Section 3.1 で導入した量子子消去 $\forall i$ が本質的に用いられる—あの操作の出番がここである。本書では規則の存在を認めるにとどめ、詳細は Cohen et al. (2018) の §6.2 に譲る。また、universe type_i 自身の Kan 構造（型どうしの path に沿った $\text{hcomp} \cdot \text{transp}$ ）も Glue 型を用いて与えられる（同 §7.1）。以下ではこれらを既知として用いる。

Definition 89 (Univalence Map). $\Gamma \vdash e : A \simeq B$ に対して：

$$\text{ua}(e) \stackrel{\text{def}}{=} \langle i \rangle \text{Glue}[(i = 0) \mapsto (A, e), (i = 1) \mapsto (B, \text{idEquiv}(B))] B$$

$$: \text{Path type}_i A B$$

端点を検算しよう。 $[0/i]$ では面公式 $(i = 0)$ が $1_{\mathbb{F}}$ となるから、第 1 の ∂ -等値性と Lemma 28 により

$$\text{Glue}[1_{\mathbb{F}} \mapsto (A, e)] B =_{\partial} A$$

すなわち $\text{ua}(e) 0 = A$ である。同様に $[1/i]$ では $(i = 1)$ が $1_{\mathbb{F}}$ となり $\text{ua}(e) 1 = B$ である。よって $\text{ua}(e)$ はたしかに A から B への型の path である（ $\text{Path type}_i A B$ の形成には、 type_i 自身が type_{i+1} の元であるという本書の sort の階層を用いている）。

さらに、 $\text{ua}(e)$ に沿った移送は e の関数成分の適用に一致する：任意の $a : A$ に対して

$$\vdash \text{Path } B (\text{transp}^i (\text{ua}(e) i) 0_{\mathbb{F}} a) (\pi_1(e) a) \text{ true}$$

が成り立つ ($\text{ua}\beta$; Glue の transp の κ -簡約から従う。判断的等値性としては成り立たない)。すなわち、「同値で作った path に沿って運ぶ」ことは「同値を適用する」ことに path の意味で等しい。

univalence の証明の準備として、2 つの補題を用意する。第一の補題は、可縮性を Section 3.2 の語彙—部分元と延長可能性—で言い換えるものである。「どの部分元も延長できる」とは、幾何学的には「どんな開いた形の穴も埋まる」ことであり、それが「1 点に潰れる」ことと同じだ、というのがこの補題の内容である。

Lemma 90 (Contractibility as Extensibility). 型 $\Gamma \vdash C : \text{type}$ について、次は同値である：(i) $\text{isContr}(C)$; (ii) 任意の extent φ 上の任意の部分元 $\Gamma, \varphi \vdash u : C$ が延長可能である。

Proof. (i) $\Rightarrow$ (ii) : (c_0, h) を可縮性の witness とする ($c_0 : C, h : (y : C) \rightarrow \text{Path } C c_0 y$)。部分元 u に対して

$$a \stackrel{\text{def}}{=} \text{hcomp}_C^i [\varphi \mapsto h u i] c_0$$

とおく。底面接続は、 φ 上で $h u 0 =_{\partial} c_0$ (端点規則) による。 ∂ -等値性により φ 上で $a =_{\partial} h u 1 =_{\partial} u$ だから、 a が u を延長する。

(ii) $\Rightarrow$ (i) : まず extent $0_{\mathbb{F}}$ 上の空 system $[]$ は部分元であり (Definition 27 の $n = 0$ の場合)、その延長として中心 $c_0 : C$ を得る。次に任意の $y : C$ に対し、文脈 $\Gamma, i : \mathbb{I}$ における extent $(i=0) \vee (i=1)$ 上の部分元 $[(i=0) c_0, (i=1) y]$ の延長 a をとれば、 $\langle i \rangle a : \text{Path } C c_0 y$ である (Section 3.2 の末尾で述べた「部分元の延長として path を見る」例そのものである)。□

第二の補題は、Glue 型の除去写像そのものについての定理である。

Theorem 91 (Unglue is an Equivalence). $\Gamma \vdash \varphi : \mathbb{F}, \Gamma \vdash A : \text{type}, \Gamma, \varphi \vdash (T, e) : (e : T \simeq A)$ に対し、 $B \stackrel{\text{def}}{=} \text{Glue} [\varphi \mapsto (T, e)] A$ とおくと、 $\text{unglue} : B \rightarrow A$ は同値である。

証明 : $a : A$ を固定し、 $\text{isContr}(\text{fib}(\text{unglue}, a))$ を Lemma 90 (ii) で示す。extent ψ 上の部分元 $\Gamma, \psi \vdash (t, p) : \text{fib}(\text{unglue}, a)$ ($t : B, p : \text{Path } A (\text{unglue } t) a$) をとり、これを延長する全域元を 3 段で構成する。

(1) φ の上で fiber の可縮性を使う。 φ 上では $B =_{\partial} T$ かつ $\text{unglue} =_{\partial} \pi_1(e)$ (∂ -規則) だから、 (t, p) の $\varphi \wedge \psi$ への制限は $\text{fib}(\pi_1(e), a)$ の部分元である。 isEquiv の定義によりこの fiber は可縮だから、Lemma 90 (i) $\Rightarrow$ (ii) を制限文脈 Γ, φ で適用して、延長 $\Gamma, \varphi \vdash (t_1, p_1) : \text{fib}(\pi_1(e), a)$ ($\psi \wedge \varphi$ 上で (t, p) に等しい) を得る。

(2) A の中で蓋を作る。

$$a_1 \stackrel{\text{def}}{=} \text{hcomp}_A^i [\psi \mapsto p(1-i), \varphi \mapsto p_1(1-i)] a$$

とおく。底面接続は $p1 =_{\partial} a$ 、 $p_1 1 =_{\partial} a$ (端点規則)、重なり $\psi \wedge \varphi$ 上の両立は (1) の $p = p_1$ による。 ∂ -等値性により

$$a_1 =_{\partial} p0 =_{\partial} \text{unglue } t \quad (\psi \text{ 上}) \quad a_1 =_{\partial} p_1 0 =_{\partial} \pi_1(e) t_1 \quad (\varphi \text{ 上})$$

(3) 貼り合わせる。(2) の第 2 の等値性は (GlueI) の第 3 前提そのものだから、

$$t' \stackrel{\text{def}}{=} \text{glue} [\varphi \mapsto t_1] a_1 : B$$

が型付く。 ψ 上では、 η により $t = \text{glue} [\varphi \mapsto t] (\text{unglue } t)$ であり、 $t_1 =_{\partial} t$ ($\varphi \wedge \psi$ 上) と $a_1 =_{\partial} \text{unglue } t$ (ψ 上) から $t' =_{\partial} t$ を得る。また β により $\text{unglue } t' \rightarrow_{\beta} a_1$ である。最後に、(2) の hcomp の充填 (Remark 33) $Q(j) \stackrel{\text{def}}{=} \text{hfill}_A^i [\psi \mapsto p(1-i), \varphi \mapsto p_1(1-i)] a j$ を反転して $p' \stackrel{\text{def}}{=} \langle j \rangle Q(1-j)$ とおくと、 $p'0 = Q(1) = a_1 = \text{unglue } t'$ 、 $p'1 = Q(0) = a$ であり、 ψ 上では $Q(j) =_{\partial} p(1-i)[j/i] = p(1-j)$ から $p'j = pj$ 、すなわち p' は ψ 上で p に一致する。

こうして (t', p') が (t, p) を (成分ごとに) 延長する。Lemma 90 により fiber は可縮であり、 unglue は同値である。□

Corollary 92. 任意の $\Gamma \vdash A : \text{type}_i$ に対し、型 $(X : \text{type}_i) \times (X \simeq A)$ は可縮である。

証明：Lemma 90 (ii) $\Rightarrow$ (i) による。 $C \stackrel{\text{def}}{=} (X : \text{type}_i) \times (X \simeq A)$ とおき、extent φ 上の部分元 $\Gamma, \varphi \vdash (T, f) : C$ をとる。全域元の候補は、まさに Glue 型が与える：

$$(B^*, u^*) \stackrel{\text{def}}{=} (\text{Glue} [\varphi \mapsto (T, f)] A, (\text{unglue}, \text{Theorem 91 の witness}))$$

φ 上では $B^* =_{\partial} T$ かつ $\text{unglue} =_{\partial} \pi_1(f)$ (∂ -規則) だから、第 1 成分と関数成分はちょうど一致する。**isEquiv** の witness 成分は一致するとは限らないが、**isEquiv**(g) は命題である (**isContr** が命題であること—The Univalent Foundations Program 2013 の補題 3.11.4—と Lemma 61 による) から、 φ 上で $q : \text{Path } C (B^*, u^*) (T, f)$ が第 1・第 2 成分定値の path として得られる。そこで $\text{hcomp}_C^i [\varphi \mapsto q i] (B^*, u^*)$ が (T, f) をちょうど φ 上で一致する形で延長する。よって C は可縮である。□

Theorem 93 (Univalence). 任意の $A, B : \text{type}_i$ に対して、 transp を用いて構成される標準的な写像 $\text{pathToEquiv} : \text{Path type}_i A B \rightarrow (A \simeq B)$ は同値である。すなわち、以下が成り立つ。

$$\vdash (\text{Path type}_i A B) \simeq (A \simeq B) \text{ true}$$

証明：本質は Corollary 92 である。まず、 $(X : \text{type}_i) \times \text{Path type}_i X A$ も可縮である：Section 2.2 の singleton の可縮性の構成 (connection による) は型に依らないから、宇宙の上でもそのまま成り立つ。 pathToEquiv は各 X ごとの写像だから、2 つの全空間の間の写像

$$(X : \text{type}_i) \times \text{Path type}_i X A \longrightarrow (X : \text{type}_i) \times (X \simeq A)$$

を誘導する。両辺はともに可縮であり、可縮な型の間の任意の写像は同値である (往復の合成は、可縮性の与える path によりいずれも恒等に結ばれる)。そして、全空間の上で同値である fiberwise 写像は各 fiber でも同値である (The Univalent Foundations

Program 2013 の定理 4.7.7) から、各 X で `pathToEquiv` は同値である。逆写像を具体的に与えるのが `ua` であること (`uaβ` と `isEquiv` の命題性による) も同様に確かめられる。以上は Cohen et al. (2018) §7.2 の証明 (延長可能性・`unglue` の同値性・ Σ の可縮性) を本書の記法に書き起こしたものである。□

homotopy type theory では univalence は公理として仮定される (The Univalent Foundations Program 2013) のに対し、cubical type theory ではこのように **定理**であり、しかも `ua(e)` は Glue 型の具体的な項なので、それに沿った移送が (κ -簡約により) 計算する。これが「univalence の構成的解釈」の意味である。

5.1 An Alternative Integer Type

Remark 80 で見たとおり、`trunc` をもつ $\mathbb{Z}$ からの型値再帰 (universe への除去) は素朴には通らない—universe は集合ではないからである。本節では、`trunc` をもたない整数型の別定義を与え、(1) それでも集合になることが **定理**として証明できること、(2) universe への除去が直接できること、を見る。そのうえで univalence により 2 つの定義を結び、片方の成果をもう片方へ **移送**する。

Definition 94 (The Integer Type $\mathbb{Z}_c$). 高次帰納型 $\mathbb{Z}_c$ は、パラメータなしで、次の 2 つの構成子により定義される：

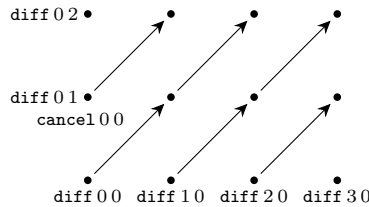
$$\frac{\Gamma \vdash a : \mathbb{N} \quad \Gamma \vdash b : \mathbb{N}}{\Gamma \vdash \text{diff } a b : \mathbb{Z}_c} \quad \frac{\Gamma \vdash a : \mathbb{N} \quad \Gamma \vdash b : \mathbb{N} \quad \Gamma \vdash r : \mathbb{I}}{\Gamma \vdash \text{cancel } a b r : \mathbb{Z}_c}$$

ただし `cancel` は境界の等値性判断

$$\text{cancel } a b 0 =_{\partial} \text{diff } a b \quad \text{cancel } a b 1 =_{\partial} \text{diff } (s(a)) (s(b))$$

をとまなう (`hcomp` 構成子も定義に含まれる)。

$\mathbb{Z}$ (Definition 57) との違いは 2 点：`path` 構成子が「同値関係のすべての witness」ではなく **生成元 1 本** (各 (a, b) に対して $\langle a, b \rangle \sim_d \langle s(a), s(b) \rangle$ だけ) に絞られていること、そして `trunc` が**ない**ことである。幾何学的には、 $\mathbb{Z}_c$ は格子 $\mathbb{N} \times \mathbb{N}$ の各点に `point` を置き、対角線方向の辺 `cancel` で結んだグラフであり、各連結成分 (= 1 つの整数) は「半直線」状で、**閉路をもたない**：



$\mathbb{Z}$ で `trunc` が必要だった理由 (Definition 57 の脚注：平行辺と合成辺の作る閉路) が、ここでは最初から生じないのである。その代償として、一般の同値 $\langle m, n \rangle \sim_d \langle p, q \rangle$ の witness は構成子 1 回では得られず、`cancel` を鎖状に継いだ合成 (`hcomp`) として構成することになる。

recursion principle には、**行き先への条件が付かない**：型 C への関数は、 $d : \mathbb{N} \rightarrow \mathbb{N} \rightarrow C$ と、`cancel` の行き先 $cl : (a, b : \mathbb{N}) \rightarrow \text{Path } C (d a b) (d (s(a)) (s(b)))$ を与えれば定義できる (S^1 と同じ形である)。この「条件なし」こそが、以下で universe への除去を可能

にする。

■ $\mathbb{Z}_c$ は集合である (定理として) Section 4.7 の norm と Int をそのまま使う。 $\text{toInt}_c : \mathbb{Z}_c \rightarrow \text{Int}$ は $\text{toInt}_c(\text{diff } a b) \stackrel{\text{def}}{=} \text{norm}(a, b)$ で定義され、 $\text{cancel } a b$ の場合の義務は $\text{norm}(a, b)$ と $\text{norm}(s(a), s(b))$ を結ぶ path だが、 norm の第 3 等式によりこれは判断的に同一の項であり、定値 path でよい。したがって $\text{toInt}_c(\text{cancel } a b i)$ は i に依存しない— well-definedness が判断的計算に吸収される、 cancel 版に特有の現象である ($\mathbb{Z}$ の toInt では Lemma 75 という命題的な path を要した)。 fromInt_c は $\iota_1(n) \mapsto \text{diff } n 0$, $\iota_2(m) \mapsto \text{diff } 0 (s(m))$ とする。往復の一致のうち $\text{toInt}_c \circ \text{fromInt}_c$ は (Theorem 76 (i) と同様) 判断的。逆向き $m_{a,b}^c : \text{Path } \mathbb{Z}_c (\text{fromInt}_c(\text{norm}(a, b))) (\text{diff } a b)$ は二重帰納法

$$m_{a,0}^c \stackrel{\text{def}}{=} 1 \quad m_{0,s(b)}^c \stackrel{\text{def}}{=} 1 \quad m_{s(a),s(b)}^c \stackrel{\text{def}}{=} m_{a,b}^c \cdot \langle i \rangle \text{cancel } a b i$$

で構成できるが、今度は trunc がいないため、 cancel に沿った整合性 ($m_{a,b}^c$ と $m_{s(a),s(b)}^c$ を結ぶ正方形) を手で与える必要がある。求める正方形は、合成 $m_{a,b}^c \cdot \langle i \rangle \text{cancel } a b i$ の充填、すなわち Remark 33 の hfill がそのまま与える (立方体の詳細は機械検証 IntCancel モジュールを参照。この「 hcomp で構成したものの整合性は hfill が保証する」という手筋が cancel 版の証明の基調である)。こうして $\mathbb{Z}_c \simeq \text{Int}$ が成り立ち、集合性をレトラクトに沿って引き戻すことで (あるいは等値性の決定可能性と Hedberg の定理により) :

Theorem 95 ($\mathbb{Z}_c$ is a Set). $\mathbb{Z}_c \simeq \text{Int}$ であり、 $\text{isSet}(\mathbb{Z}_c)$ が成り立つ。

$\mathbb{Z}$ では公理 (構成子 trunc) だったものが、 $\mathbb{Z}_c$ では定理である。この対比を、一般的な戦術として述べておこう。

Remark 96 (Sets First or Sets Later). 高次元の整合性 (正方形・立方体……) をすべて手で構成するのは煩雑であり、しばしば非現実的である。2つの戦術がある。第一は、 set truncation の構成子を最初から定義に加え、整合性の義務を Lemma 60 に委ねる道—Definition 57 の $\mathbb{Z}$ がこれである。除去に「行き先は集合」という条件が付く代わりに、証明は一様に軽くなる。第二は、生成元を絞って truncation なしで型を定義し、決定可能な等値性をもつ既知の型との同値を経由して「実は集合であった」ことを後から証明する道—本節の $\mathbb{Z}_c$ がこれである。除去は無条件であり universe への除去も直接できるが、定義や証明に現れる高次の整合性は (集合性が確立するまで) 手で埋める。

■ $\mathbb{Z}_c$ 上の型値再帰 それでは Remark 80 の (c) の道を実行しよう。述語 $P : \mathbb{Z}_c \rightarrow \text{type}$ を $\text{recursion principle}$ で定義するには、 $\text{cancel } a b$ の場合に型どうしの path $\text{Path type } (P(\text{diff } a b)) (P(\text{diff } (s(a)) (s(b))))$ を与えればよく、それ以外の義務はない。 ua がこれを可能にする。例として「零である」という述語を定義する。

まず、 s が path の同値を誘導することを見る。 $a, b : \mathbb{N}$ に対して

$$\text{sucPathEquiv}(a, b) : (\text{Path } \mathbb{N} a b) \simeq (\text{Path } \mathbb{N} (s(a)) (s(b)))$$

を構成する。関数成分は $f \stackrel{\text{def}}{=} \lambda p. \langle i \rangle s(p i)$ 、逆写像は前者関数 pd (Lemma 77) を用いて $g \stackrel{\text{def}}{=} \lambda q. \langle i \rangle \text{pd}(q i)$ とする ($q i$ の端点は $s(a), s(b)$ だから、 $g(q)$ の端点は a, b である)。往復のうち $g(f(p)) = \langle i \rangle \text{pd}(s(p i)) = p$ は判断的に成り立つ。もう一方の $f(g(q))$ と q は、同じ端点をもつ $\mathbb{N}$ の path どうしであるから、 $\mathbb{N}$ が集合であること (Section 4.7) により path で結ばれる。互いに逆な写像の組であるから、Definition 87 の後に注意した事実により同値が得られる。

これを用いて：

$$\begin{aligned}\text{isZero}_c(\text{diff } a b) &\stackrel{\text{def}}{=} \text{Path } \mathbb{N} a b \\ \text{isZero}_c(\text{cancel } a b i) &\stackrel{\text{def}}{=} \text{ua}(\text{sucPathEquiv}(a, b)) i\end{aligned}$$

(hcomp の場合は universe の hcomp—Glue により与えられる—による。) $\text{ua}(\text{sucPathEquiv}(a, b))$ の端点はまさに $\text{Path } \mathbb{N} a b$ と $\text{Path } \mathbb{N} (\text{s}(a)) (\text{s}(b))$ であるから、recursion principle の証明義務が満たされている。こうして $\text{isZero}_c(\text{diff } 0 0) = \text{Path } \mathbb{N} 0 0$ は inhabited、 $\text{isZero}_c(\text{diff } (\text{s}(n)) 0) = \text{Path } \mathbb{N} (\text{s}(n)) 0$ は ($\mathbb{N}$ の等値性の決定可能性より) empty である。正值性 $\mathbb{Z}_c^+$ や順序 $<$ も、 $\mathbb{N}$ 上の対応する述語と「 s による同値」を与えれば、まったく同じ手筋で定義できる。

■2つの整数型を結ぶ—移送 Theorem 76 と Theorem 95 の同値を合成すると

$$e : \mathbb{Z} \simeq \text{Int} \simeq \mathbb{Z}_c \quad \text{したがって} \quad \text{ua}(e) : \text{Path type}_i \mathbb{Z} \mathbb{Z}_c$$

を得る。異なる構成子で定義された2つの高次帰納型が、型として等しいのである。この path の上を、成果が両方向に行き来する。

第一に、述語は $\mathbb{Z}$ へ運べる。 $\text{isZero}_{\mathbb{Z}} \stackrel{\text{def}}{=} \lambda z. \text{isZero}_c(\pi_1(e) z)$ と合成すれば、 $\mathbb{Z}$ 上の「零である」述語が得られる (正值性・順序も同様)。これが Remark 80 の (c) の道の完成である。 $\mathbb{Z}$ 上で直接には定義できなかった型値再帰が、trunc のない $\mathbb{Z}_c$ を経由することで実現された。

第二に、構造は $\mathbb{Z}_c$ へ運べる。Section 4.6 で証明した $(\mathbb{Z}, +_{\mathbb{Z}})$ の法則たちを、演算ごと束ねて移送する：motive を

$$M \stackrel{\text{def}}{=} \lambda X. (\text{add} : X \rightarrow X \rightarrow X) \times \mathbf{Assoc}(X, \text{add})$$

$$\mathbf{Assoc}(X, \text{add}) \stackrel{\text{def}}{=} (x, y, w : X) \rightarrow \text{Path } X (\text{add} (\text{add } x y) w) (\text{add } x (\text{add } y w))$$

(「加法とその結合律」の型) とすると、subst (Exercise 43) を $\text{ua}(e)$ に沿って適用するだけで、 $M(\mathbb{Z})$ の元 $(+_{\mathbb{Z}}, \text{Theorem 70 の証明})$ から $M(\mathbb{Z}_c)$ の元

$$t \stackrel{\text{def}}{=} \text{subst}(\text{ua}(e), (+_{\mathbb{Z}}, \text{Theorem 70 の証明}))$$

が得られる： $\mathbb{Z}_c$ は、何も証明し直すことなく、結合律付きの加法 $\pi_1(t)$ を手に入れる。交換律・単位元・逆元も同じ Σ に積めばよい。これが univalence の実用上の意味である：同値な型は等しく、等しい型の上では、定義も定理も代入原理で流用できる。

Exercise 97. (1) $\mathbb{Z}_c$ 上に加法 $+_c$ を直接、recursion principle で定義せよ。第1引数の cancel には cancel で応えられる ($\text{s}(m) + n = \text{s}(m + n)$ が判断的なため) が、第2引数の cancel では端点が addSucc の分だけずれるため、cancel と $\mathbb{N}$ の補題の path を側面にもつ hcomp が必要。その開いた箱を書き下せ。(2) 移送で得た加法 $\pi_1(t)$ と (1) の $+_c$ が path で結ばれること (一致すること) を、 $\pi_1(e)$ が加法を保つこと (準同型性) に帰着させて示せ。(ヒント：point 構成子上で確かめれば、集合性 (Theorem 95) が残りを埋める。)

2つの定義の損得をまとめる。 $\mathbb{Z}$ (eq + trunc、商型スタイル) は、古典的な商の構成と1対1に対応し、証明は一樣に軽く、同じ設計が $\mathbb{Q} \cdot \mathbb{R}$ へ反復できる。代わりに除去には「行き先は集合 (示すものは命題)」の条件が付き、universe への除去は直接にはで

きない。 $\mathbb{Z}_c$ (cancel、生成元スタイル) は、truncation が不要で除去が無条件、型値再帰が直接できる。代わりに well-definedness の外の整合性 (正方形・充填) を集合性が確立するまで手で埋めることになり、生成元の取り方は successor の構造に依存した個別の工夫である。univalence は、この 2 つを「同じ型の 2 つの提示」として自由に往復させる。

History and Further Reading

Glue 型と univalence の証明は Cohen et al. (2018) の §6–7 による (本章の提示は $\text{hcomp} / \text{transp}$ 分解に合わせて調整した)。Voevodsky による univalence 公理の定式化と simplicial set モデル、およびそれが構成的モデルの探求を促した経緯については、Chapter 1 と Chapter 3 の History を参照。

関数外延性や商型を構成的に扱う別の系譜として、observational type theory (Altenkirch et al., 2007) および setoid によるアプローチ (Hofmann, 1995) がある。cubical type theory は、これらが個別に扱ってきた問題 (関数外延性・商・univalence) を単一の interval の機構で一挙に解決する体系とみることができる。cubical 手法全般の survey としては Mörtberg (2021) がある。

Bibliography

- Altenkirch, T., C. McBride, and W. Swierstra. (2007) “Observational equality, now!”, In *Proceedings of the 2007 Workshop on Programming Languages meets Program Verification (PLPV '07)*, A. Stump and H. Xi (eds.). pp.57–68.
- Angiuli, C., G. Brunerie, T. Coquand, R. Harper, K.-B. Hou (Favonia), and D. R. Licata. (2021) “Syntax and models of Cartesian cubical type theory”, *Mathematical Structures in Computer Science* **31**(4), pp.424–468.
- Angiuli, C., R. Harper, and T. Wilson. (2017) “Computational higher-dimensional type theory”, In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. pp.680–693, ACM.
- Awodey, S. and M. A. Warren. (2009) “Homotopy theoretic models of identity types”, *Mathematical Proceedings of the Cambridge Philosophical Society* **146**(1), pp.45–55.
- Bekki, D. (2014) “Representing Anaphora with Dependent Types”, In *Proceedings of Logical Aspects of Computational Linguistics (8th international conference, LACL2014, Toulouse, France)*, *LNCS 8535*, N. Asher and S. V. Soloviev (eds.). pp.14–29, Springer, Heidelberg.
- Bekki, D. and K. Mineshima. (2017) “Context-Passing and Underspecification in Dependent Type Semantics”, In: S. Chatzikyriakidis and Z. Luo (eds.): *Modern Perspectives in Type-Theoretical Semantics*, Vol. 98 of *Studies in Linguistics and Philosophy*. Springer, pp.11–41.
- Bezem, M., T. Coquand, and S. Huber. (2014) “A Model of Type Theory in Cubical Sets”, In *Proceedings of 19th International Conference on Types for Proofs and Programs (TYPES 2013)*, R. Matthes and A. Schubert (eds.), Vol. 26 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.107–128, Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- Cavallo, E. and R. Harper. (2019) “Higher Inductive Types in Cubical Computational Type Theory”, *Proceedings of the ACM on Programming Languages* **3**(POPL), pp.1:1–1:27.
- Cohen, C., T. Coquand, S. Huber, and A. Mörtberg. (2018) “Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom”, In *Proceedings of 21st International Conference on Types for Proofs and Programs (TYPES 2015)*, T. Uustalu (ed.), Vol. 69 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.5:1–5:34, Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Preprint: arXiv:1611.02108.
- Coquand, T., S. Huber, and A. Mörtberg. (2018) “On Higher Inductive Types in Cubical Type Theory”, In *Proceedings of LICS '18: Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*. pp.255–264.
- Hofmann, M. (1995) “Extensional concepts in intensional type theory”, Ph.D. thesis, University of Edinburgh.
- Hofmann, M. and T. Streicher. (1998) “The groupoid interpretation of type theory”,

- In: G. Sambin and J. M. Smith (eds.): *Twenty-Five Years of Constructive Type Theory (Venice, 1995)*, Oxford Logic Guides, vol.36. New York, Oxford University Press.
- Huber, S. (2019) “Canonicity for Cubical Type Theory”, *Journal of Automated Reasoning* **63**(2), pp.173–210.
- Kapulkin, K. and P. L. Lumsdaine. (2021) “The simplicial model of Univalent Foundations (after Voevodsky)”, *Journal of the European Mathematical Society* **23**(6), pp.2071–2126.
- Lumsdaine, P. L. and M. Shulman. (2020) “Semantics of higher inductive types”, *Mathematical Proceedings of the Cambridge Philosophical Society* **169**(1), pp.159–208.
- Martin-Löf, P. (1975) “An intuitionistic theory of types”, In: H. E. Rose and J. Shepherdson (eds.): *Logic Colloquium ’73*. Amsterdam, North-Holland, pp.73–118.
- Martin-Löf, P. (1984) *Intuitionistic Type Theory*, Vol. 17. Naples, Italy: Bibliopolis. Sambin, Giovanni (ed.).
- Mörtberg, A. (2021) “Cubical methods in homotopy type theory and univalent foundations”, *Mathematical Structures in Computer Science* **31**(10), pp.1147–1184.
- Nordström, B., K. Petersson, and J. Smith. (1990) *Programming in Martin-Löf’s Type Theory*. Oxford University Press.
- Nordström, B., K. Petersson, and J. M. Smith. (2000) “Martin-Löf’s Type Theory”, In: *Handbook of Logic in Computer Science, Vol 5*. Oxford University Press.
- Orton, I. and A. M. Pitts. (2016) “Axioms for Modelling Cubical Type Theory in a Topos”, In *Proceedings of 25th EACSL Annual Conference on Computer Science Logic (CSL 2016)*, J.-M. Talbot and L. Regnier (eds.), Vol. 62 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.24:1–24:19, Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Journal version: *Logical Methods in Computer Science* 14(4:23), 2018.
- Sterling, J. and C. Angiuli. (2021) “Normalization for Cubical Type Theory”, In *Proceedings of 36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2021)*. pp.1–15, IEEE.
- Streicher, T. (1993) “Investigations into Intensional Type Theory”, Ph.D. thesis, Habilitationsschrift, Ludwig-Maximilians-Universität.
- The Univalent Foundations Program. (2013) *Homotopy Type Theory: Univalent Foundations of Mathematics*. <https://homotopytypetheory.org/book>.
- Vezzosi, A., A. Mörtberg, and A. Abel. (2019) “Cubical Agda: A Dependently Typed Programming Language with Univalence and Higher Inductive Types”, *Proceedings of the ACM on Programming Languages* **3**, pp.87:1–87:29.